Intelligent Discovery of Photos in an Event through Face Recognition: A Selfie-Driven, Liveness-Gated Retrieval System

Nandalal Shukla
[Department / Programme], [Institution]
[City, Country]
shuklanandalal@gmail.com

***Abstract*—Event photographers deliver thousands of pictures, yet each guest wants only the few in which they appear. Manual sorting does not scale, and one shared album exposes every guest's photos to anyone who holds the link. This paper presents PhotoMatch AI, an end-to-end system in which an organizer uploads an album once and each guest retrieves a private gallery of their own photos by sending a single selfie to a Telegram or WhatsApp bot. Faces are detected with SCRFD, described by 512-dimensional ArcFace embeddings and compared by cosine similarity inside PostgreSQL with the pgvector extension, which also serves as the job queue, so no separate vector database or message broker is required. For private events, an optional liveness gate combines a guided head turn, a same-person check, a passive anti-spoofing ensemble and a match against the original selfie before any photo is shown. We evaluate the shipped implementation on a four-core CPU. In a controlled nine-identity perturbation study the embeddings stayed above the 0.45 decision threshold under strong blur, compression, darkness, noise and rotation, and no false match occurred in 3,280 cross-identity comparisons (largest similarity 0.238). Exact event-scoped search took 2.1 ms for 1,000 faces and 24 ms for 10,000 faces and was almost independent of table size (16 to 23 ms when the table grew tenfold), whereas an HNSW index retrieved at most 93% of the true matches at 250,000 faces. We state plainly what is not yet evaluated: standard face benchmarks and a labelled presentation-attack protocol are left for future work.**

***Index Terms*—face recognition, photo retrieval, liveness detection, presentation attack detection, vector search, pgvector, ArcFace**

I. Introduction

A wedding, a conference or a college festival produces thousands of photographs within a few hours. A few days later the album is ready, and the organizer becomes a human search engine: relatives and friends write to ask for "the pictures with me". Two workarounds are common, and both are unsatisfying. Sending photos one person at a time consumes hours of the organizer's attention, while publishing a single album link delivers everything to everyone and gives guests no way to keep their image private. The underlying task is simple to state—*given one picture of a person, find that person in a large collection*—and it is exactly the task that deep face recognition now solves with high reliability on public benchmarks [1]–[4].

Commercial event-photo platforms already advertise selfie-based search, in one case with a liveness check in front of the search [5], [6]. To our knowledge there is no peer-reviewed description of how such a service is built or how it behaves, so its design trade-offs and failure modes remain undocumented. Building one raises four practical questions. *(i) Cost shape:* indexing is bursty (tens of thousands of faces within hours, then idle), so infrastructure should be light enough to run on a single small server. *(ii) Privacy:* face embeddings are biometric templates [7], [8], and a forwarded gallery link must not reveal photos to the person who received it. *(iii) Reach:* guests are non-technical and will not install an application or create an account, so the interaction must happen in the messengers they already use. *(iv) Usability of liveness:* a liveness check that fails real people at the exact moment they want their photos is worse than none.

Contributions. This paper describes and evaluates PhotoMatch AI, an open implementation built to answer those questions:

1) an architecture that keeps relational data, face vectors and the job queue in *one* PostgreSQL database, uploads photos directly from the browser to object storage, and runs all neural networks in a stateless, CPU-only service;
2) an *event-scoped exact* cosine search, justified empirically against HNSW indexing, including a post-filtering failure mode that we reproduce;
3) a liveness-gated delivery protocol that binds a gallery link to the live face of the person who sent the selfie, using a guided head-turn challenge with burst capture, a same-person check, a passive anti-spoofing ensemble and identity matching;
4) a dependency-light ONNX Runtime port of the detection, alignment, embedding and head-pose stages that installs from binary wheels on Windows, macOS and Linux; and
5) a measurement study of cost, robustness and scale on commodity hardware, with an explicit account of what the evidence does *not* show.

Section II reviews related work, Section III formalizes the task, Sections IV–VI present the system, Section VII reports the evaluation, and Sections VIII and IX discuss limits and conclude.

II. Related Work

A. Deep face recognition

Early recognition relied on holistic projections such as Eigenfaces [9] and hand-crafted texture descriptors such as local binary patterns [10]. Deep networks changed the picture: DeepFace reported 97.35% verification accuracy on Labeled Faces in the Wild (LFW) [1], [11], VGG-Face showed that a standard deep network trained on a carefully curated dataset is competitive [12], FaceNet learned a compact Euclidean embedding with a triplet loss and reached 99.63% [2], and margin-based softmax losses—SphereFace, CosFace and ArcFace—made embeddings more discriminative on the hypersphere [3], [13]–[15]. Residual networks [16], large training sets such as WebFace260M [17] and efficient large-scale training [18] followed; comprehensive surveys are available [4]. LFW is now close to saturation, so harder protocols such as IJB-C are used to compare modern models [19], [20]. Lightweight networks for mobile deployment [21] show that accuracy and cost can be traded, but our setting is a server that processes an album in the background and can afford a ResNet-50 recognizer.

B. Face detection, alignment and head pose

Cascades [22] gave way to multi-task convolutional detectors [23], single-shot detectors with landmark supervision [24] and compute-efficient designs such as SCRFD [25], evaluated on WIDER FACE [26]. Event photographs contain many small faces, which motivates scale-aware detection [27]. Recognition networks expect faces aligned to a canonical template, which is obtained from detected landmarks with a similarity transform [28]. Head pose can be derived from dense or 3-D landmarks [29], [30]; we use it to verify a head-turn challenge.

C. Liveness and presentation attack detection

Presentation attack detection (PAD) is standardized in ISO/IEC 30107-3 [31]. Early software countermeasures looked for involuntary motion such as eye blinks [32] or the motion of face parts [33], or for texture differences between real skin and prints and screens [34], [35]. Datasets such as CASIA-FASD, OULU-NPU and CelebA-Spoof enabled deep PAD with auxiliary depth or remote-photoplethysmography supervision [36]–[39] and central-difference convolutions [40]; a survey is given in [41]. Passive models generalize imperfectly across cameras and attack types [41], which is why challenge–response and passive checks are often combined. Digital manipulation and injection attacks, which bypass the camera, are a separate and growing threat [42]. We use the MiniFASNet ensemble released under the Apache-2.0 license by Minivision [43] and pair it with an active head-turn challenge.

D. Person search in photo collections

Research on personal photo albums addresses recognition when faces are small, occluded or turned away [44], [45] and groups unlabeled faces by clustering [46]. Our problem is narrower and query-driven: a guest supplies one frontal selfie and the system must return every photo of that person in one event, with privacy controls that photo-organizing tools do not need.

E. Vector search

Approximate nearest-neighbor methods such as HNSW graphs [47], product quantization [48] and GPU libraries [49] make billion-scale search feasible. The `pgvector` extension brings exact and HNSW search into PostgreSQL [50]. Recent releases add iterative scans because index scans combined with a filter can return too few rows [51]; we measure that effect in Section VII.

F. Privacy and fairness

Deep templates can be inverted to recover recognizable face images [7], so stored embeddings are sensitive. Biometric data is a special category under the GDPR [8]. Recognition error rates differ across demographic groups [52], [53]. We return to these points in Section VIII.

III. Problem Statement and Design Goals

Let an event E consist of photos $P = \{p_1, \dots, p_N\}$. A detector finds a set of faces $F(p)$ in each photo, and an embedding function ϕ maps an aligned face crop to a unit vector in $\mathbb{R}^{512}$. A guest submits a selfie s whose single main face yields the query $q = \phi(s)$. The system returns

$$R(q) = \Big\{ p \in P_{\text{approved}} \;:\; \max_{f \in F(p)} q^{\top} \phi(f) \;\geq\; \tau \Big\}, \qquad (1)$$

ranked by the maximum similarity, with $\tau = 0.45$ in the deployed configuration. Only the best face per photo matters, because a guest appears in a photo at most once. Photos contributed by third parties are excluded from $R(q)$ until the organizer approves them.

We derived six goals from the scenario in the introduction. **G1 Privacy:** selfies are never persisted, galleries are reachable only through a per-search link, and an organizer can switch guest access off at any time. **G2 Scale:** thousands of photos and tens of thousands of faces per event. **G3 Low operating cost:** one small server, no GPU, as few moving parts as possible. **G4 Reach:** Telegram and WhatsApp, no app or account for guests. **G5 Optional liveness:** for private events, only the person who sent the selfie may open the gallery. **G6 Portability:** installable on a student laptop without a C++ toolchain.

IV. System Architecture

A. Components

Figure 1 shows the system. A React single-page application serves organizers and guests. A Node.js API (Express 5, Prisma, better-auth) handles authentication, events, signed uploads, bot webhooks and gallery access, and serves the compiled web application in production so that browser and API share one origin. A background worker consumes jobs from a queue implemented with pg-boss [54] on PostgreSQL. A stateless Python service (FastAPI with ONNX Runtime [55]) is the only component that runs neural networks and never touches the database. PostgreSQL stores users and sessions,

events, photos, face vectors (`vector(512)`) and the queue. Photo files live in an external object store (Cloudinary).

Consolidating state in PostgreSQL has three consequences. Deleting an event removes its photos and faces through `ON DELETE CASCADE`, so biometric data cannot outlive the event (G1). Queue and data share one backup and one transaction boundary, which makes saving the faces of a photo and marking it ready a single atomic transaction. And no Redis or dedicated vector database has to be installed, secured and monitored (G3).

B. Ingestion pipeline

Figure 2 shows the path of a photo. The browser first computes a SHA-256 digest [56] of each file and asks the API which digests the event already holds; known files are skipped, even when renamed, and a unique database constraint on (event, digest) enforces the same rule on the server. New files are uploaded directly from the browser to the object store with a short-lived signature that is bound to the event's folder, so large files never traverse the API. Uploaded photos are registered in batches of 50 and one `index-photo` job is enqueued per photo. The worker downloads a copy limited to 1,600 pixels in width, calls the AI service, and stores all returned embeddings together with the photo status in one transaction. Failed jobs are retried three times with exponential back-off, which absorbs short outages of the AI service.

C. Guest search through messengers

An event is identified by an eight-character code drawn from 31 unambiguous symbols ($31^8 \approx 8.5 \times 10^{11}$ codes). The guest page opens WhatsApp or Telegram with the code prefilled; the guest sends a selfie (Fig. 3). Telegram webhooks are authenticated with a secret header and WhatsApp webhooks with an HMAC-SHA-256 signature compared in constant time. A shared conversation module implements the dialogue once, while the two platform adapters only parse incoming messages and send replies. The selfie is processed in memory; only its embedding is stored on the search record, because the liveness gate needs it later (Section VI). The reply contains a link to the guest's private gallery.

D. Organizer features

Organizers sign in with e-mail (verified) or Google. They can invite co-organizers, enable a contributor upload link whose photos stay hidden until approved, switch guest search and liveness on or off, and delete the event. The query in (1) therefore filters on the approval flag.

V. Face Analysis and Matching

A. Detection

SCRFD (`det_10g`) [25] receives the photo scaled to fit a 640×640 canvas with its aspect ratio preserved. It predicts, at strides 8, 16 and 32 with two anchors per grid point, a face score, box distances and five landmarks. Detections with score ≥ 0.5 are decoded and merged by non-maximum suppression (IoU 0.4).

B. Alignment and embedding

The five landmarks are mapped to the ArcFace template by a least-squares similarity transform [28], the face is warped to 112×112 pixels, and an ArcFace ResNet-50 trained on WebFace600K produces a 512-dimensional vector that is L2-normalized [3], [57]. Because vectors have unit length, cosine similarity is a dot product, and PostgreSQL's cosine distance operator <=> returns $1 - q^\top \phi(f)$.

C. An ONNX Runtime port

The reference `insightface` Python package compiles C++ extensions during installation, which fails on many Windows machines (goal G6). We therefore re-implemented the pre- and post-processing around the published ONNX files with only ONNX Runtime, OpenCV and NumPy. On 11 test images the port found the same faces and boxes as the reference package and produced embeddings with cosine similarity ≥ 0.999999 to it; yaw estimates agreed within about 2°.

D. Selfie handling

A selfie must contain one clear main face. If a second face has at least 60% of the main face's area, the system refuses to guess whom the guest means and asks for another picture. Smaller background faces are ignored.

E. Exact event-scoped search

For a query q, the database evaluates (1) (Fig. 4) for the rows of one event: the `event_id` index selects its faces, the distance operator scores them, a threshold keeps the matches, and `GROUP BY photo` keeps the best face per photo. The cost is $O(n_E\, d)$ for n_E faces in the event and $d = 512$, and, as measured in Section VII, nearly independent of the number of other events. We chose this exact strategy over an approximate index because events contain at most a few tens of thousands of faces and because approximate indexes can silently miss matches (Section VII); a guest who is told "no photos found" when photos exist is a failure the design should avoid.

VI. Liveness-Gated Private Delivery

A. Threat model

For a private event, the organizer wants to prevent three things: (a) a recipient who forwards or leaks a gallery link, (b) a person who holds up another person's printed photograph or a screen to the camera, and (c) a person who submits somebody else's photograph as the original selfie in order to obtain that person's photos. Presentation attacks that inject synthetic video into the camera pipeline, high-quality 3-D masks and collusion with the genuine guest are out of scope.

B. Protocol

Figure 5 summarizes the protocol. The browser guides the guest through tips, a face-in-oval alignment step and a ready button, followed by a three-second countdown. It then captures a burst of 4 frames while the guest looks straight at the camera and a burst of 12 frames, 300 ms apart, while the guest turns

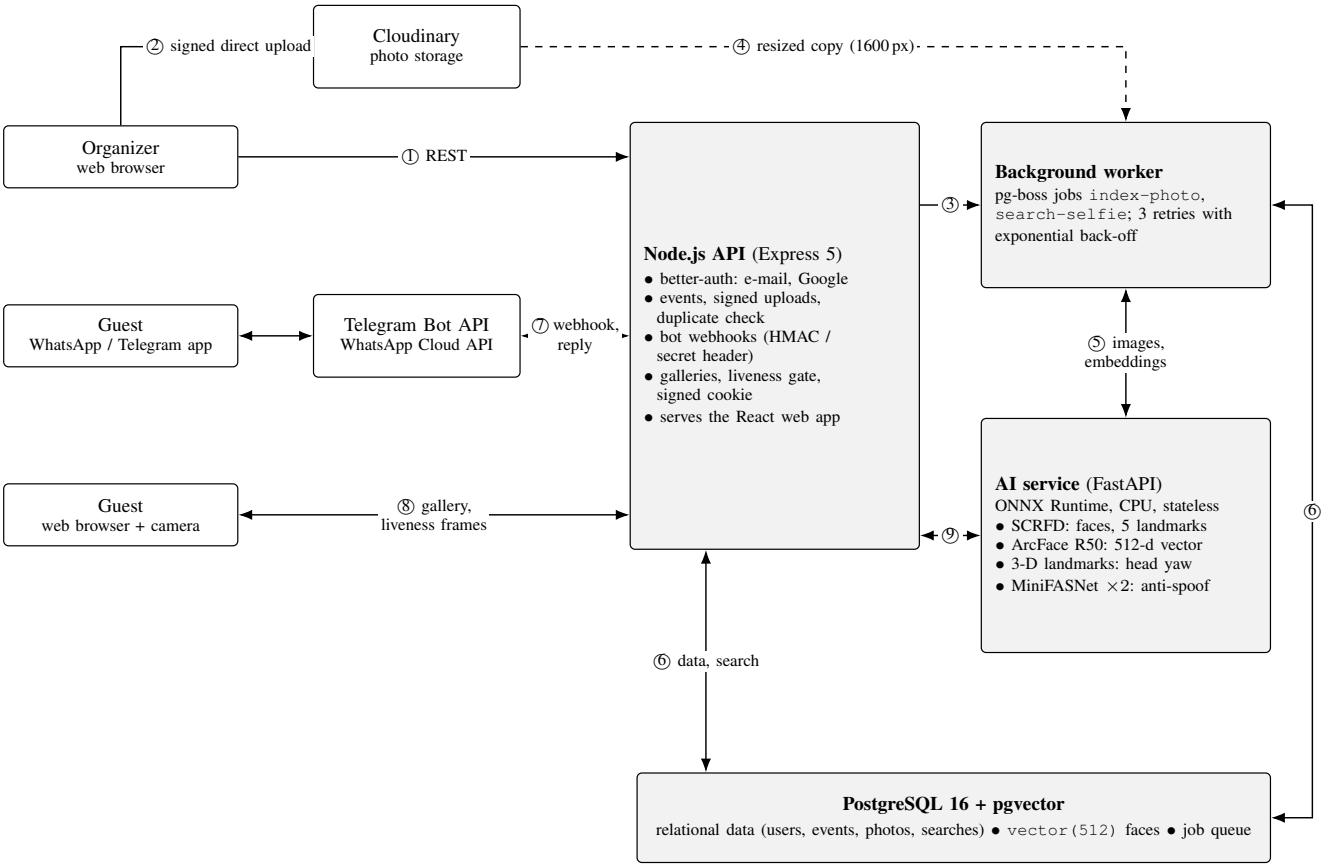


Fig. 1. Architecture of PhotoMatch AI. Numbers follow the text: (1)–(4) ingestion, (7) guest messaging, (8)–(9) gallery access with optional liveness, (5)–(6) analysis and storage.

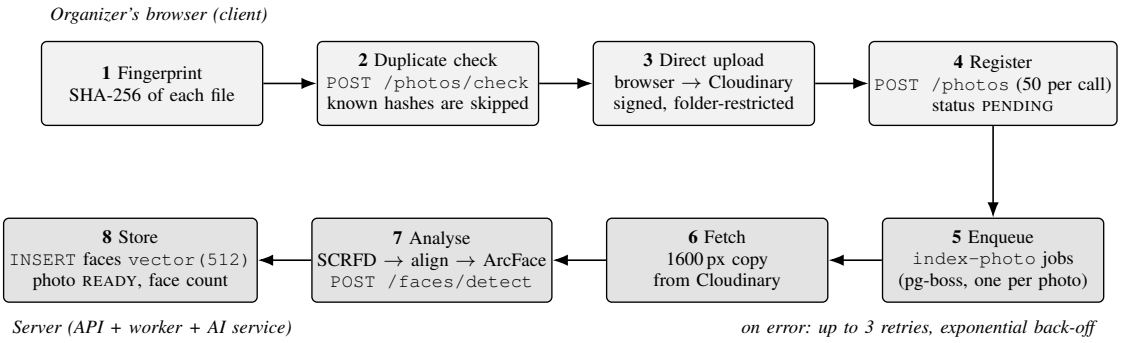


Fig. 2. Photo ingestion pipeline. Steps 1–4 run in the organizer's browser, steps 5–8 on the server.

their head. A first prototype that captured one frame at the end of a countdown rejected real users who turned slowly. Burst capture removes the timing requirement: the server keeps the most frontal frame and the most turned frame of the two bursts and ignores the rest.

C. Checks

Head turn. A 3-D 68-landmark network yields the line between the outer eye corners; seen from the front it lies in the image plane, and a turned head gives it depth. The yaw is the angle between the line and the image plane,

$$\theta = \operatorname{atan2}\big(\Delta z,\ \sqrt{\Delta x^2 + \Delta y^2}\big), \tag{2}$$

which is insensitive to head tilt. The straight frame must satisfy $|\theta_f| \le 15^\circ$ and the turned frame must differ by at least 20°. A printed photograph cannot satisfy the second condition. **Same person.** The embeddings of the two frames must have cosine similarity ≥ 0.3; this bar is lower than the match threshold because a turned face looks less similar. It prevents showing one person's face in one burst and another's in the second. **Anti-spoofing.** Two MiniFASNet networks (V2 with a $2.7\times$

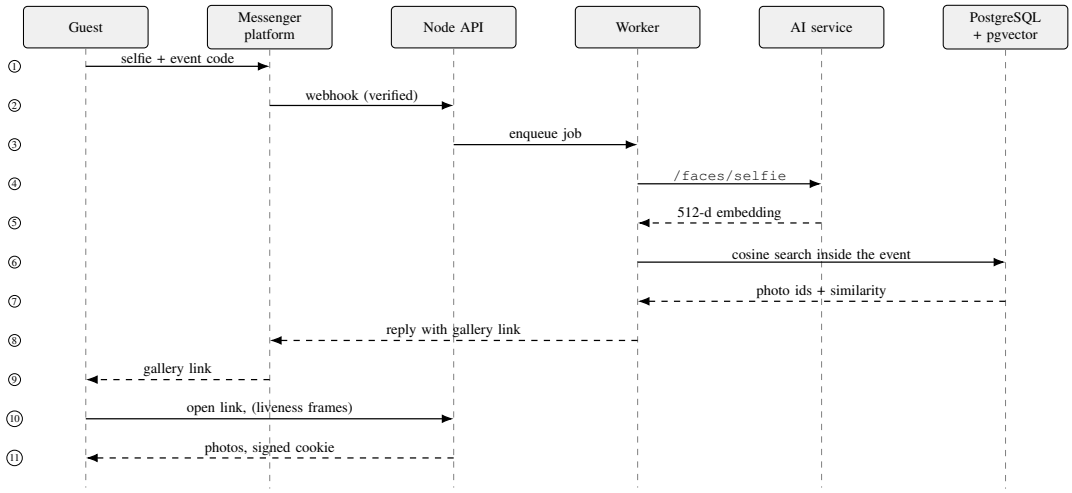


Fig. 3. Guest search sequence. Steps 10–11 show the visit to the gallery link, with liveness frames when the event requires them.

```
SELECT f.photo_id,
       MAX(1 - (f.embedding <=> :q))  AS similarity
FROM   face f JOIN photo p ON p.id = f.photo_id
WHERE  f.event_id = :event AND p.approved
  AND  1 - (f.embedding <=> :q) >= :tau   -- 0.45
GROUP  BY f.photo_id
ORDER  BY similarity DESC  LIMIT 1000;
```

Fig. 4. The matching query of (1): exact cosine search restricted to one event (simplified from the production code).

crop and V1SE with a $4.0\times$ crop around the face, both 80×80 input) output a probability that the face is real [43]. The two outputs are averaged per frame, and the smaller value of the two frames must be ≥ 0.7. **Identity.** Finally, the API compares the live embedding with the selfie embedding stored on the search; the cosine similarity must reach $\tau = 0.45$. This step closes threat (c): an attacker who supplies a victim's photograph as the selfie cannot later pass the camera check with their own face.

D. Session handling

After a pass the API sets a signed, `httpOnly` cookie that is valid for 30 minutes on that browser only. Each gallery link allows five attempts; further attempts are refused until the guest sends a new selfie. Liveness requests are rate limited. All thresholds are configuration values.

VII. Experimental Evaluation

A. Scope and setup

We evaluate the shipped code, not a research prototype. Measurements ran on a virtual machine with four cores of an Intel Xeon processor at 2.30 GHz and 15 GiB of memory (Linux 6.18, no GPU), with Python 3.12, ONNX Runtime 1.23.2, OpenCV 4.12, PostgreSQL 16.15 and pgvector 0.6.0. Latencies are medians over repeated runs after warm-up with the 5th and 95th percentiles; all raw results are in the repository (`paper/data`). *No public identity-labelled benchmark could be downloaded in our build environment, so we did not re-measure recognition accuracy on LFW, IJB-C or a presentation-attack dataset.* Instead, Section VII-B cites the accuracy reported for the underlying models by their authors, and the remaining subsections measure properties that do not need a large labelled dataset: cost, stability of the embedding under controlled degradations, behaviour at scale, and functional correctness. The limits of this design are discussed in Section VIII.

TABLE I
Accuracy on LFW as reported by the original authors, and the reported results of the model pack used in this work. Not measured by us. The buffalo_l pack also reports CFP-FP 99.33, AgeDB-30 98.23 and IJB-C (E4) 97.25 [57].

Method	Source	LFW acc. (%)
DeepFace	[1]	97.35
FaceNet	[2]	99.63
SphereFace (CASIA-WebFace)	[13]	99.42
CosFace	[14]	99.73
buffalo_l (SCRFD + ArcFace R50)	[57]	**99.83**

B. Reported accuracy of the underlying models

Table I lists published accuracies. These numbers come from the cited sources, were obtained on different training sets and protocols, and are not our measurements; they indicate the capability of the model family we deploy rather than the performance of our port, which we verified only for equivalence to the reference implementation (Section V-C).

C. Cost of the analysis pipeline

Figure 6(a) breaks down the cost on a six-face photo scaled to 1,600 px (the size the worker sends). The detector takes 66.9 ms per photo and ArcFace 64.6 ms per face, whereas JPEG decoding (5.6 ms), head pose (14.0 ms) and the two anti-spoofing networks (6.1 ms) are small. Whole-photo analysis took 175.8 ms for a one-face selfie and 508.7 ms for the six-face photo, consistent with $t \approx 0.11\,\mathrm{s} + 0.067\,\mathrm{s} \times n_{\mathrm{faces}}$.

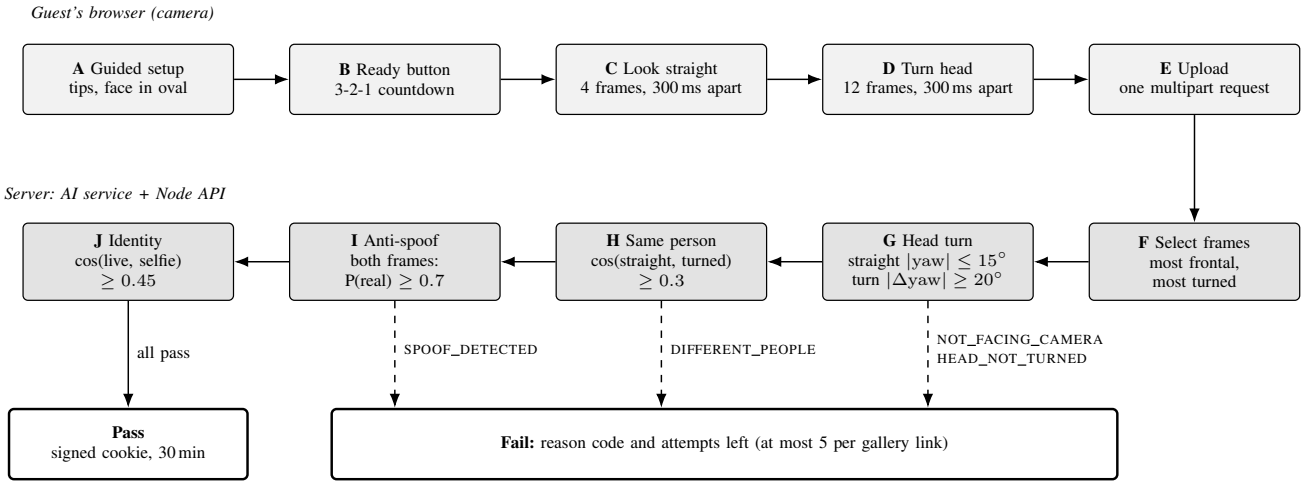


Fig. 5. Liveness protocol. The client captures two bursts; the server selects two frames and applies five checks in order. Any failing check returns a machine-readable reason and consumes one of five attempts.

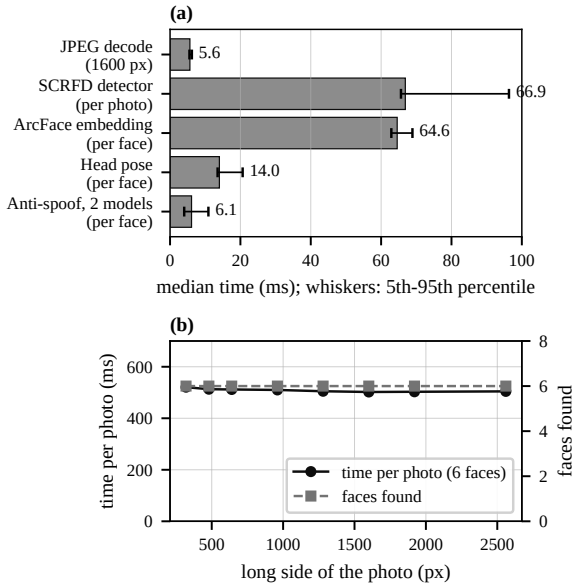


Fig. 6. (a) Median cost of each pipeline stage at 1,600 px. (b) Time per photo and faces found for one six-face photo at different resolutions.

Extrapolating this model, indexing a 5,000-photo event with four faces per photo would take about 31 minutes in one process; we did not run an end-to-end indexing job of that size. Figure 6(b) shows that cost does not depend on the input resolution because the detector input is fixed at 640×640 and the recognizer sees 112×112 crops: all six faces were found at every size from 320 to 2,560 px, with the smallest face 24 px wide at 320 px.

D. Stability of the embedding under degradation

Design. We collected nine identities: the six people of a public group photograph, a sample selfie and two public portraits. Each image was scaled so that the face is 160 px wide, one degradation at a time was applied to the whole image, the face was detected again, and its embedding was compared with the clean embedding of the same identity. The same perturbed embedding was also compared with the clean embeddings of the eight other identities. This is a stress test of the pipeline's stability, *not* a measure of recognition accuracy across poses, ages or sessions.

Reference values. Clean embeddings of different identities had mean similarity 0.016 ± 0.067 (36 pairs, maximum 0.197), whereas three different photographs of one person scored 0.73–0.78. The threshold $\tau = 0.45$ lies in the gap between the two groups.

Results. Figure 7 shows the similarity to the clean face. All nine faces were detected in every condition of the six sweeps and stayed above τ up to Gaussian blur $\sigma = 6$ px (mean 0.65), JPEG quality 8 (mean 0.83), brightness reduced to 8% (mean 0.84), noise $\sigma = 45$ gray levels (mean 0.75), rotation up to 45° (mean 0.92), and 45% of the lower face covered (mean 0.66). Falling below τ occurred at the next severity: blur $\sigma = 8$ (7 of 9 accepted), noise $\sigma = 60$ (8 of 9), rotation 60° (8 of 9, minimum 0.40) and 60% lower-face coverage (3 of 9). Across the 3,280 comparisons between a perturbed face and a different identity, the largest similarity was 0.238 and none reached τ.

Small faces. Group photographs contain small faces. We placed each identity at decreasing sizes inside a 1,600 px-wide canvas (Fig. 8). Down to 36 px all nine faces were detected and accepted; at 28 px 8 of 9 were detected, and at 20 px only 6 of 9 were detected and 5 accepted. Faces smaller than about 30 px in a 1,600 px photo should therefore be considered unreliable. This is consistent with the observation that tiny faces need dedicated treatment [27].

E. Vector search

Method. We generated synthetic embeddings so that cost and recall could be measured at sizes for which no labelled data exists. Each event has identity clusters of eight unit vectors in $\mathbb{R}^{512}$ with mutual cosine similarity between about 0.36 and 0.81; different identities are random directions. A

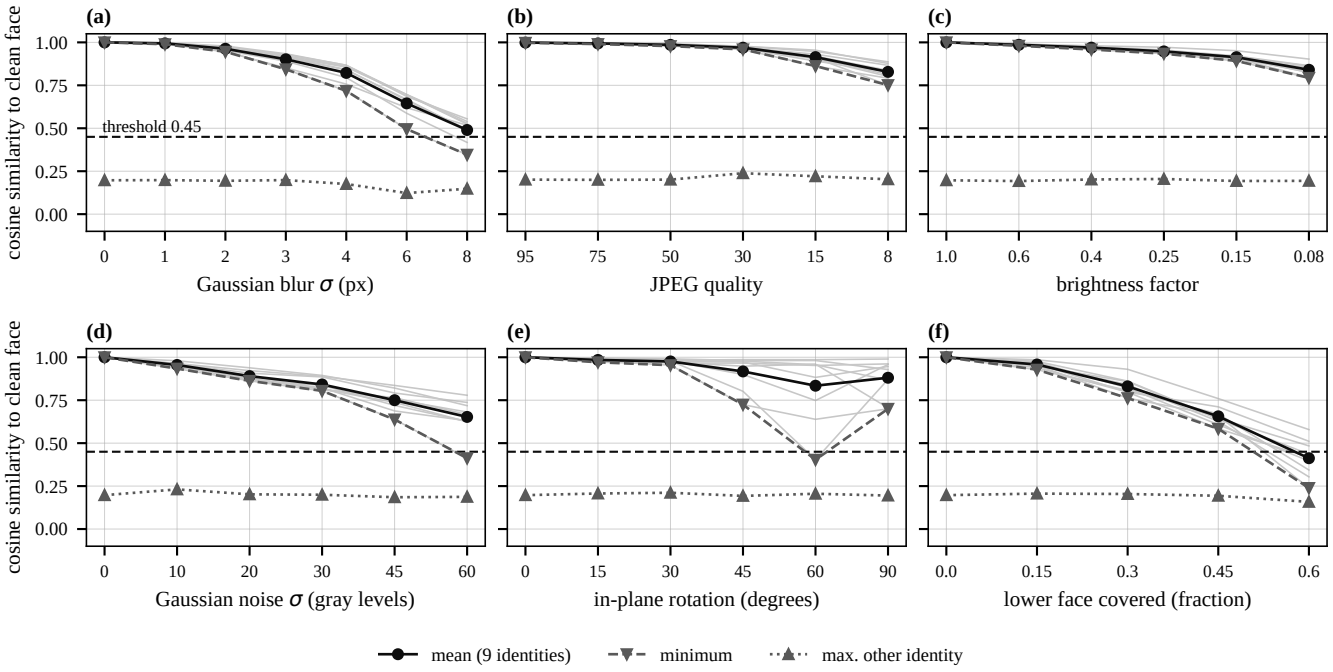


Fig. 7. Cosine similarity between a degraded face and the clean face of the same identity (thin lines: nine identities; thick: mean) and the largest similarity to any other identity (dotted). The dashed line is the decision threshold 0.45.

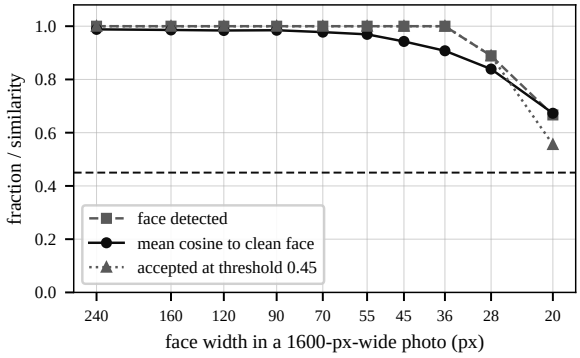


Fig. 8. Effect of face size in a 1,600 px-wide photo on detection, similarity and acceptance (nine identities).

TABLE II
GLOBAL TOP-100 SEARCH OVER SYNTHETIC EMBEDDINGS: EXACT SCAN VERSUS HNSW (MEDIAN LATENCY; RECALL OF FACES WITH SIMILARITY ≥ 0.45; 100 QUERIES PER TABLE FOR HNSW, 30 FOR THE EXACT SCAN).

Faces	Exact ms	HNSW, ef=100 ms	HNSW, ef=100 recall	HNSW, ef=400 ms	HNSW, ef=400 recall	Index MB
10,000	14.7	2.0	1.000	4.9	1.000	26
50,000	102.0	4.1	0.993	11.7	0.998	130
100,000	230.6	7.1	0.958	20.3	0.985	260
250,000	395.8	10.1	0.826	28.6	0.928	651

query is a noisy copy of an identity (cosine 0.8 to the identity centre). Each event query returns about eight photos on average. Queries used the production SQL, including the join with the approval flag.

Exact event-scoped search. Figure 9(a) shows median latency growing close to linearly with event size: 1.1 ms for 500 faces, 2.1 ms for 1,000, 23.6 ms for 10,000 and 244 ms for 100,000 faces. Panel (b) shows that a 10,000-face event took 16.2 ms in a 50,000-face table and 22.9 ms in a 500,000-face table, so a tenfold larger table raised the latency by about 41% while the event size was unchanged: the cost is governed mainly by the size of the event rather than by the total number of stored faces.

HNSW versus exact. Table II compares a global HNSW index ($m = 16$, `ef_construction`=64) with exact scans of the whole table. HNSW is 5 to 40 times faster than the exact scan but loses matches as the table grows. At 250,000 faces, recall of the matching set was 0.66, 0.83, 0.89 and 0.93 for `ef_search` 40, 100, 200 and 400, and some individual queries retrieved none of their true matches. Recall improves with `ef_search` and falls with table size (Fig. 9(c)). Random high-dimensional clusters are a hard case for graph indexes, so real embeddings may do better, but the experiment shows why we do not depend on an approximate index for a feature whose failure mode is "your photos are missing".

HNSW with an event filter. PhotoMatch restricts every query to one event. We tested a filtered HNSW query on the 250,000-face table, which holds 250 events. With the `event_id` index present, the planner chose an exact plan on its own (recall 1.00, 2.2–2.7 ms for a 1,000-face event). When we removed that index inside a rolled-back transaction to force the HNSW plan, the filter was applied after the index scan: for 100 requested rows the query returned on average

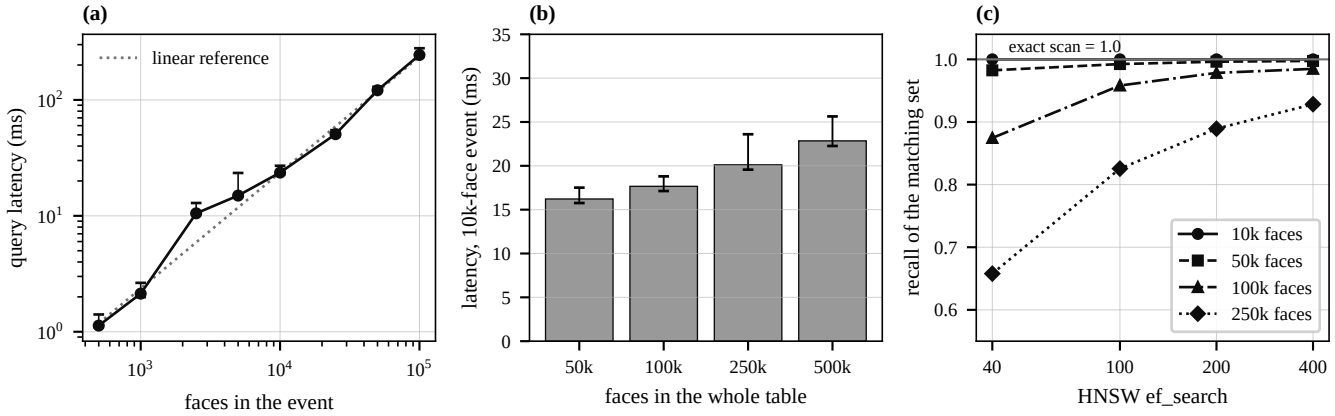


Fig. 9. (a) Exact event-scoped query latency versus event size (log–log; whiskers 5th–95th percentile). (b) Latency for a 10,000-face event versus total table size. (c) Recall of the matching set for a global HNSW index versus `ef_search` and table size.

5.5, 7.3 and 9.4 rows and recalled 0.66, 0.85 and 0.94 of the true matches for `ef_search` 40, 100 and 400. This post-filtering behaviour is the reason newer pgvector releases add iterative scans [51]; we used release 0.6.0, so the experiment documents the failure mode rather than the current behaviour of the extension.

F. Liveness stage

Sanity check on two samples. We scored the two public samples released with the MiniFASNet project, a live selfie and a phone photograph of a printed poster [43]. The live selfie obtained P(real) of 0.9999 (V2) and 0.99999 (V1SE), average 0.99996; the print obtained 0.0044 and 0.305, average 0.155, below the 0.7 threshold. This is a smoke test with $n = 2$ and in-distribution samples; it is *not* an estimate of the attack-presentation or bona fide classification error rates defined in ISO/IEC 30107-3 [31], which require a labelled protocol such as OULU-NPU [37] or CelebA-Spoof [38].

Genuine users in poor conditions. Applying the same degradations to the live selfie, the average P(real) stayed at or above 0.89 for brightness down to 8%, blur up to $\sigma = 6$ px, JPEG quality down to 8 and face widths from 320 to 36 px (minimum 0.893 at $\sigma = 6$); no genuine sample would have been rejected. With a single identity this says little about the false-rejection rate across devices and skin tones.

Cost. The anti-spoofing networks cost 4.6 ms per face. Selecting the frames of one complete attempt (16 copies of a 640×480 frame, standing in for 4 + 12 webcam frames; detection with head pose on every frame and embedding of the two selected frames) took 2.16 s (5th–95th percentile 2.04–2.27 s) in the AI service, and the two anti-spoofing evaluations add about 9 ms. This is the wait the guest experiences in addition to network time.

G. Throughput of the AI service

We sent the six-face photograph at 1,600 px to `POST /faces/detect` from concurrent clients (48 requests per configuration). One process delivered 1.99, 1.96 and 2.00 photos/s with 1, 2 and 4 clients: throughput is flat because

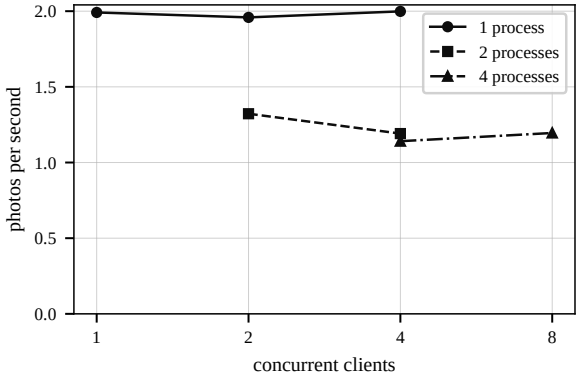


Fig. 10. Throughput of the AI service for a six-face 1,600 px photograph versus concurrent clients and server processes (four CPU cores).

the service serializes inference with a lock and each ONNX Runtime session already uses all four cores, so extra clients only add queueing delay (median latency 0.50, 1.02 and 2.00 s). Starting more server processes *lowered* throughput (1.32 photos/s with two processes and two clients, 1.14–1.20 with four), which we attribute to thread oversubscription on four cores; we did not test per-process thread limits. On this hardware the practical way to scale indexing is therefore more machines, not more processes. A one-face selfie costs about 0.18 s of AI time, but because guest searches share this service with indexing, a guest who writes while a large album is being indexed waits behind the backlog; a separate service instance or queue priority for selfies would remove that coupling (not implemented).

H. Functional testing

Three automated suites guard the implementation. A harness with 42 end-to-end checks exercises the real API, worker, database and AI models with the external messengers and object store replaced by local stand-ins: sign-up and e-mail verification, photo indexing, Telegram and WhatsApp flows including signature verification, correct matches, each liveness rejection reason, forged cookies, attempt lockout, duplicate

photos and organizer controls. It passed 42 of 42 checks on the final code; it predates the co-organizer and contributor-link features, whose access rules are covered by the API's 18 unit tests (event progress, event codes, message parsing, webhook signatures, access rules and e-mail escaping). The AI service has 15 tests with the real models, including the live selfie versus printed photograph check. All three suites pass.

VIII. Discussion and Limitations

What the evidence supports. The measurements support three engineering claims: face analysis is cheap enough for a CPU-only server (about 0.11 s plus 0.067 s per face); exact event-scoped search is fast enough for realistic events and, unlike a global HNSW index, never misses matches; and the embedding stage is stable under the degradations typical of event photography, with a wide margin between same-person and different-person similarity in our sample.

What it does not support. First, recognition accuracy was not re-measured on a public benchmark; the numbers in Table I are those of the original authors. Our robustness study uses nine identities and cannot reveal demographic differences, which are known to exist [52], [53], nor errors across ages, expressions or extreme poses. Second, the vector-search experiment uses synthetic embeddings; absolute recall on real face data may differ. Third, the liveness evidence consists of two samples and a quality sweep on one live selfie; we make no claim about APCER, BPCER or resistance to video replay on screens, which passive models are known to find hard [41]. The head-turn requirement blocks static photographs but not a replayed video of the genuine guest turning their head, nor injection attacks [42]. Fourth, all timings come from one virtual machine.

Privacy and law. Embeddings are biometric templates and can be inverted to approximate face images [7]. PhotoMatch stores no selfie image, ties all face data to the event and deletes it with the event, but a public deployment would still need explicit consent, a retention policy, a privacy notice and an assessment under laws such as the GDPR [8]. The recognition models in the `buffalo_l` pack are licensed by their authors for non-commercial research use [57], which is acceptable for a research or hobby deployment but would have to be replaced or licensed for commercial use.

Design lessons. (1) For bounded collections, exact search with a relational filter is simpler and more reliable than an approximate index. (2) Putting the queue in the database removed a service without a measurable cost at this scale, but the single-process AI service, not the database, is what limits indexing speed. (3) Capturing bursts and choosing frames on the server turned an unreliable liveness step into one guests can complete at their own pace.

IX. Conclusion and Future Work

PhotoMatch AI lets a guest obtain their photos from an event album with one selfie, optionally protected by a liveness gate that binds the gallery to the person who sent the selfie. The system keeps data, vectors and jobs in a single PostgreSQL database and runs its neural networks on a CPU. Our measurements show that this design is fast, that its embedding stage is stable under common degradations, and that exact event-scoped search avoids the recall losses of approximate indexes; they also show where it stops being reliable, namely faces below roughly 30 px, extreme blur or occlusion, and horizontally scaled indexing. Future work, in order of importance, is: evaluating the full pipeline on LFW, IJB-C and an event-like labelled collection with demographic breakdowns; a presentation-attack evaluation following ISO/IEC 30107-3 on OULU-NPU and CelebA-Spoof, including replay attacks; re-testing filtered HNSW with iterative scans in current pgvector releases for events beyond 100,000 faces; per-process thread limits for the AI service; face clustering to build per-person albums automatically [46]; and replacing the non-commercial models with commercially licensed ones.

Acknowledgment

The author used an AI assistant (Claude, Anthropic) for software development, running the experiments, literature search and drafting of this manuscript. The source code, experiment scripts and raw measurements are available in the project repository. [Add funding and further acknowledgments as appropriate.]

References

[1] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2014, pp. 1701–1708.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 815–823.

[3] J. Deng, J. Guo, J. Yang, N. Xue, I. Kotsia, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 5962–5979, Oct. 2022.

[4] M. Wang and W. Deng, "Deep face recognition: A survey," *Neurocomputing*, vol. 429, pp. 215–244, 2021.

[5] Kamero, "Kamero: AI photo sharing for events and weddings," Vendor website, 2026, [Online]. Available: https://kamero.ai/. Accessed: Oct. 2026.

[6] Picsolv, "Picsolv: Find your event photos with AI face recognition," Vendor website, 2026, [Online]. Available: https://picsolv.com/en. Accessed: Oct. 2026.

[7] G. Mai, K. Cao, P. C. Yuen, and A. K. Jain, "On the reconstruction of face images from deep face templates," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, pp. 1188–1202, 2019.

[8] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation)," Official Journal of the European Union, vol. L 119, pp. 1–88, May 2016.

[9] M. Turk and A. Pentland, "Eigenfaces for recognition," *J. Cogn. Neurosci.*, vol. 3, no. 1, pp. 71–86, 1991.

[10] T. Ahonen, A. Hadid, and M. Pietikäinen, "Face description with local binary patterns: Application to face recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 12, pp. 2037–2041, Dec. 2006.

[11] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled faces in the wild: A database for studying face recognition in unconstrained environments," Univ. Massachusetts Amherst, Amherst, MA, USA, Tech. Rep. 07-49, Oct. 2007.

[12] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep face recognition," in *Proc. Brit. Mach. Vis. Conf. (BMVC)*, 2015, pp. 41.1–41.12.

[13] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "SphereFace: Deep hypersphere embedding for face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 212–220.

[14] H. Wang, Y. Wang, Z. Zhou, X. Ji, D. Gong, J. Zhou, Z. Li, and W. Liu, "CosFace: Large margin cosine loss for deep face recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 5265–5274.

[15] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4690–4699.

[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[17] Z. Zhu, G. Huang, J. Deng, Y. Ye, J. Huang, X. Chen, J. Zhu, T. Yang, J. Lu, D. Du, and J. Zhou, "WebFace260M: A benchmark unveiling the power of million-scale deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 10 492–10 502.

[18] X. An, J. Deng, J. Guo, Z. Feng, X. Zhu, J. Yang, and T. Liu, "Killing two birds with one stone: Efficient and robust training of face recognition CNNs by partial FC," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 4032–4041.

[19] B. Maze, J. Adams, J. A. Duncan *et al.*, "IARPA Janus Benchmark-C: Face dataset and protocol," in *Proc. Int. Conf. Biometrics (ICB)*, 2018.

[20] G. B. Huang and E. Learned-Miller, "Labeled faces in the wild: Updates and new reporting procedures," Univ. Massachusetts Amherst, Amherst, MA, USA, Tech. Rep. UM-CS-2014-003, 2014.

[21] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for accurate real-time face verification on mobile devices," in *Proc. Chin. Conf. Biometric Recognit. (CCBR)*, ser. Lecture Notes in Computer Science, vol. 10996. Springer, 2018.

[22] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, 2001, pp. 511–518.

[23] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Process. Lett.*, vol. 23, no. 10, pp. 1499–1503, Oct. 2016.

[24] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, "RetinaFace: Single-shot multi-level face localisation in the wild," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020.

[25] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, "Sample and computation redistribution for efficient face detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[26] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 5525–5533.

[27] P. Hu and D. Ramanan, "Finding tiny faces," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1522–1530.

[28] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 13, no. 4, pp. 376–380, Apr. 1991.

[29] A. Bulat and G. Tzimiropoulos, "How far are we from solving the 2D & 3D face alignment problem? (and a dataset of 230,000 3D facial landmarks)," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 1021–1030.

[30] E. Murphy-Chutorian and M. M. Trivedi, "Head pose estimation in computer vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 4, pp. 607–626, Apr. 2009.

[31] ISO/IEC, *Information technology — Biometric presentation attack detection — Part 3: Testing and reporting*, International Organization for Standardization Std. ISO/IEC 30 107-3:2017, 2017.

[32] G. Pan, L. Sun, Z. Wu, and S. Lao, "Eyeblink-based anti-spoofing in face recognition from a generic webcamera," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2007, pp. 1–8.

[33] K. Kollreider, H. Fronthaler, and J. Bigun, "Non-intrusive liveness detection by face images," *Image Vis. Comput.*, vol. 27, no. 3, pp. 233–244, 2009.

[34] I. Chingovska, A. Anjos, and S. Marcel, "On the effectiveness of local binary patterns in face anti-spoofing," in *Proc. Int. Conf. Biometrics Special Interest Group (BIOSIG)*, 2012.

[35] Z. Boulkenafet, J. Komulainen, and A. Hadid, "Face spoofing detection using colour texture analysis," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 8, pp. 1818–1830, Aug. 2016.

[36] Z. Zhang, J. Yan, S. Liu, Z. Lei, D. Yi, and S. Z. Li, "A face antispoofing database with diverse attacks," in *Proc. 5th IAPR Int. Conf. Biometrics (ICB)*, 2012, pp. 26–31.

[37] Z. Boulkenafet, J. Komulainen, L. Li, X. Feng, and A. Hadid, "OULU-NPU: A mobile face presentation attack database with real-world variations," in *Proc. IEEE Int. Conf. Autom. Face Gesture Recognit. (FG)*, 2017, pp. 612–618.

[38] Y. Zhang, Z. Yin, Y. Li, G. Yin, J. Yan, J. Shao, and Z. Liu, "CelebA-Spoof: Large-scale face anti-spoofing dataset with rich annotations," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

[39] Y. Liu, A. Jourabloo, and X. Liu, "Learning deep models for face anti-spoofing: Binary or auxiliary supervision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 389–398.

[40] Z. Yu, C. Zhao, Z. Wang, Y. Qin, Z. Su, X. Li, F. Zhou, and G. Zhao, "Searching central difference convolutional networks for face anti-spoofing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 5295–5305.

[41] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5609–5631, May 2023.

[42] R. Tolosana, R. Vera-Rodriguez, J. Fierrez, A. Morales, and J. Ortega-Garcia, "DeepFakes and beyond: A survey of face manipulation and fake detection," *arXiv preprint arXiv:2001.00179*, 2020.

[43] Minivision AI, "Silent-face-anti-spoofing," GitHub repository, Apache-2.0 license, 2020, [Online]. Available: https://github.com/minivision-ai/Silent-Face-Anti-Spoofing. Accessed: Oct. 2026.

[44] N. Zhang, M. Paluri, Y. Taigman, R. Fergus, and L. Bourdev, "Beyond frontal faces: Improving person recognition using multiple cues," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 4804–4813.

[45] S. J. Oh, R. Benenson, M. Fritz, and B. Schiele, "Person recognition in personal photo collections," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 3862–3870.

[46] Z. Wang, L. Zheng, Y. Li, and S. Wang, "Linkage based face clustering via graph convolution network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 1117–1125.

[47] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[48] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011.

[49] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[50] A. Kane, "pgvector: Open-source vector similarity search for Postgres," GitHub repository, 2024, [Online]. Available: https://github.com/pgvector/pgvector. Accessed: Oct. 2026.

[51] PostgreSQL Global Development Group, "pgvector 0.8.0 released," PostgreSQL news announcement, 2024, [Online]. Available: https://www.postgresql.org/about/news/pgvector-080-released-2952. Accessed: Oct. 2026.

[52] J. Buolamwini and T. Gebru, "Gender shades: Intersectional accuracy disparities in commercial gender classification," in *Proc. Conf. Fairness, Accountability Transparency (FAT*), PMLR 81*, 2018, pp. 77–91.

[53] P. Grother, M. Ngan, and K. Hanaoka, "Face recognition vendor test (FRVT) part 3: Demographic effects," National Institute of Standards and Technology, Tech. Rep. NISTIR 8280, Dec. 2019.

[54] pg-boss contributors, "pg-boss: Job queue for Node.js on PostgreSQL," GitHub repository, 2024, [Online]. Available: https://github.com/timgit/pg-boss. Accessed: Oct. 2026.

[55] ONNX Runtime developers, "ONNX Runtime," Open-source inference engine, 2021, [Online]. Available: https://onnxruntime.ai. Accessed: Oct. 2026.

[56] National Institute of Standards and Technology, "Secure hash standard (SHS)," NIST, Tech. Rep. FIPS PUB 180-4, Aug. 2015.

[57] InsightFace contributors, "InsightFace model zoo: the `buffalo_l` model pack," GitHub repository, 2023, [Online]. Available: https://github.com/deepinsight/insightface/blob/master/model_zoo/README.md. Accessed: Oct. 2026.